

Name: Raja Thejes

Roll no: G25AIT1125

Subject: NLU Natural Language Understanding.

Sanskrit-to-English Translation Using Seq2Seq & Attention Abstract:

Sanskrit-to-English Translation Using Seq2Seq and Attention Abstract. This project focuses on building a Neural Machine Translation (NMT) system to translate Sanskrit sentences into English. Given Sanskrit's complex grammar, I implemented a Sequence-to-Sequence (Seq2Seq) model enhanced with an Attention mechanism using PyTorch. After training the model on the provided parallel corpus, I evaluated its performance using BLEU and BERTScores, and finally generated the required submission.csv file for the test dataset.

Introduction & Objectives:

Machine translation is always tricky, but translating an ancient, highly inflected language like Sanskrit brings unique challenges. Traditional rule-based methods require massive linguistic hand-crafting, so I opted for a modern Deep Learning approach that learns patterns directly from data.

The main goals of this assignment were to:

- Clean and preprocess the Sanskrit-English parallel data.
- Build separate vocabularies and tokenize text for both languages.
- Design and train an Attention-based Seq2Seq model in PyTorch.
- Evaluate the results using both lexical (BLEU) and semantic (BERTScore) metrics.
- Predict translations for the test set and format the final submission file.

Dataset & Preprocessing Pipeline:

The dataset came split into training, development (validation), and test sets, with each row containing a unique Source ID along with the parallel sentences.

Before feeding anything into the neural network, the raw text needed a lot of cleaning. I built a preprocessing pipeline that handled:

- Stripping out weird formatting and extra whitespaces.
- Tokenizing the sentences into individual words.
- Building vocabularies and mapping words to unique integer IDs.
- Adding necessary control tokens like (for padding short sentences), (for unknown words), (start of sentence), and (end of sentence).
- Wrapping everything into custom PyTorch Dataset and DataLoader objects to handle batching smoothly during training.

Model Architecture:

The core system relies on the classic Encoder-Decoder framework, but with a crucial twist to

keep it from losing track of long sentences:

- **The Encoder:** An LSTM network that processes the input Sanskrit sentence word by word, compressing the information into hidden states.
- **The Attention Mechanism:** Instead of forcing the decoder to rely purely on the very last hidden state of the encoder, the attention layer lets the decoder look back at specific parts of the source Sanskrit sentence that are most relevant to the word it is currently translating.
- **The Decoder:** Another LSTM that takes the attention-weighted context and predicts the English translation one token at a time until it hits the marker.

Training and Evaluation:

I trained the network using the Adam optimizer and CrossEntropyLoss, implementing teacher forcing to help the model converge faster. I tracked both training and validation losses across epochs, saving the weights from the epoch that achieved the lowest validation loss to avoid overfitting.

Performance Metrics:

Evaluating the model gave some interesting insights:

- BLEU Score: 0.002608
- BERTScore: Precision: 0.7467 | Recall: 0.8186 | F1-Score: 0.7806

While the BLEU score looks incredibly low on paper, it makes sense. BLEU looks for exact n-gram word matches. Because Sanskrit has a vast vocabulary and rich

morphology, and our training data was limited, the model often chose synonyms or different phrasing. The high BERTScore F1-score (0.7806) confirms this—it uses contextual embeddings to check meaning, proving that the model actually captured the semantic intent of the sentences even when it couldn't guess the exact words.

Sample Translation & Practical Challenges:

Looking at a sample run helps illustrate what's happening under the hood:

- Sanskrit Input: ते वीर ा ा ।
- Reference English: those are brave men.
- Predicted Output: [Contained tokens]

The appearance of tokens points directly to the biggest roadblock I faced: vocabulary coverage. Because Sanskrit words change form based on case, gender, and number, many words in the test set simply never appeared in the training vocabulary.

Other major hurdles included the sheer scarcity of parallel data and the architectural limitations of basic LSTMs when trying to capture complex, long-range language rules.

Future Upgrades & Conclusion:

If I were to scale this project up, the obvious next steps would be moving away from LSTMs entirely and implementing a Transformer-based architecture (like a mini-BERT or T5).

- Using subword tokenization (like Byte Pair Encoding or SentencePiece) would also fix the token issue by breaking rare words into recognizable chunks.
- Ultimately, this project was a success. I built a functional, end-to-end Neural Machine.
- Translation pipeline from scratch using PyTorch. While the data constraints kept the lexical scores low, adding the Attention mechanism clearly helped the model grasp the underlying meaning of the text, and the final pipeline successfully generated the correctly formatted submission.csv file.